

UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas
Informáticos



UNIVERSIDAD
POLITÉCNICA
DE MADRID



Máster
Deep Learning
Universidad Politécnica de Madrid

Trabajo Fin de Máster

Sistema LLM para análisis financiero histórico
mediante generación automática de código

MÁSTER DE FORMACIÓN PERMANENTE EN DEEP LEARNING

Presentado por:

Carlos Ruiz Oyarzun

Bajo la dirección de:

Adrián Girón Jiménez

Alejandro Delgado Peribáñez

Madrid, 2026

Resumen

Este Trabajo Fin de Máster propone el desarrollo incremental de un sistema orientado al análisis financiero histórico con modelos de lenguaje. El sistema parte de una consulta estructurada, datos de mercado almacenados en CSV y metadatos como tickers o fechas. A partir de esa entrada, el prototipo valida la solicitud, utiliza un LLM para interpretar la intención, genera código Python ejecutable, valida dicho código, lo lanza en un proceso controlado y transforma la salida en una respuesta interpretable mediante APIs compatibles con OpenAI.

El MVP actual soporta seis intenciones analíticas: crecimiento de precio, comparación de activos, visión descriptiva de un activo, análisis de retornos, análisis de riesgo histórico y análisis técnico. La evaluación combina pruebas funcionales, escenarios de error controlado y una comparativa entre proveedores LLM. Los resultados muestran que el sistema puede mantener el cálculo financiero bajo control programático y utilizar modelos de lenguaje como capa principal de interpretación final.

Palabras clave: análisis financiero, modelos de lenguaje, generación de código, ejecución controlada, APIs compatibles con OpenAI.

Abstract

This Master's Thesis proposes the incremental development of a language-model-driven system for historical financial analysis. The system starts from a structured query, market data stored in CSV files and metadata such as tickers or dates. From this input, the prototype validates the request, uses an LLM to interpret the user's intent, generates executable Python code, validates that code, runs it in a controlled process and transforms the output into an interpretable response through OpenAI-compatible APIs.

The current MVP supports six analytical intents: price growth, asset comparison, descriptive view of an asset, returns analysis, historical risk analysis and technical analysis. The evaluation combines functional tests, controlled error scenarios and a comparison between several LLM providers. The results show that the system can keep financial computation under programmatic control while using language models as the main final interpretation layer.

Keywords: financial analysis, language models, code generation, controlled execution, OpenAI-compatible APIs.

Índice general

Resumen	I
Abstract	II
1. Introducción	1
1.1. Contexto y definición del problema	1
1.2. Motivación	2
1.3. Objetivos	3
1.4. Alcance	3
1.5. Estructura de la memoria	4
2. Estado de la cuestión	5
2.1. Enfoque de la revisión	5
2.2. Modelos de lenguaje en finanzas	5
2.3. LLM con herramientas y ejecución	6
2.4. Generación de código	6
2.5. Fiabilidad y trazabilidad	6
2.6. Comparativa con trabajos relacionados	7
2.7. Posicionamiento de la propuesta	7
3. Datos y material de trabajo	8
3.1. Fuente de datos y datos obtenidos	8
3.2. Catálogo de consultas y proceso de descarga	9
3.3. Estructura de los CSV	11
3.4. Análisis exploratorio de datos	12
3.5. Calidad, cobertura y limitaciones	13
3.6. Trazabilidad documental y artefactos reproducibles	13
4. Metodología	15
4.1. Enfoque de desarrollo	15
4.2. Evolución incremental del MVP	15
4.3. Criterios de validación	16

4.4.	Criterios cualitativos de calidad del análisis	16
4.5.	Niveles de profundidad del análisis	17
4.6.	Evaluación con proveedores LLM	18
4.7.	Revisión humana	18
5.	Diseño del sistema	19
5.1.	Arquitectura general	19
5.2.	Nodos del flujo	20
5.3.	Contrato del código generado	21
5.4.	Gestión de errores	22
6.	Implementación	23
6.1.	Estructura del código	23
6.2.	Repositorio y organización del trabajo	23
6.3.	Contratos de datos	24
6.4.	Pipeline LLM	25
6.5.	Control de seguridad	26
6.6.	Reparación y compactación	26
6.7.	Pruebas	27
7.	Integración de modelos de lenguaje	28
7.1.	Papel del LLM en el sistema	28
7.2.	Cliente compatible con OpenAI	28
7.3.	Configuraciones evaluadas	29
7.4.	Gestión de errores de API	29
7.5.	Gestión de credenciales	31
8.	Resultados y evaluación	32
8.1.	Objetivo de la evaluación	32
8.2.	Métricas y criterios	32
8.3.	Niveles cualitativos de salida	33
8.4.	Batería progresiva	34
8.5.	Protocolo de ejecución definitiva	34
8.6.	Resultados pendientes	35
8.7.	Limitaciones metodológicas	35
8.8.	Síntesis	35
9.	Aspectos éticos, seguridad y uso responsable	36
9.1.	Delimitación financiera del sistema	36
9.2.	Riesgos de interpretación	36
9.3.	Seguridad en la generación y ejecución de código	36

9.4. Privacidad y datos	37
9.5. Sostenibilidad, costes y recursos	37
9.6. Limitaciones de uso	37
10. Conclusiones y trabajo futuro	38
Referencias	40
A. Anexos	43
A.1. Ejecución del MVP	43
A.2. Configuración de APIs	44
A.3. Batería de consultas A/B/C	44
A.4. Query de estrés visual	46
A.5. Documentación estable	46
A.6. Plantilla de revisión manual	47
A.7. Documentación interna de apoyo	48

Índice de tablas

2.1. Comparativa entre trabajos relacionados y la propuesta del TFM. . . .	7
3.1. Datos obtenidos y finalidad de cada conjunto dentro del MVP.	9
3.2. Estructura semántica de una consulta financiera antes de su procesamiento.	11
3.3. Campos OHLCV presentes en los CSV de mercado.	11
3.4. Resumen de cobertura y calidad del conjunto de datos.	12
3.5. Muestra de cobertura temporal obtenida en el EDA.	12
3.6. Métricas exploratorias por serie utilizadas para contextualizar los casos de prueba.	13
6.1. Módulos principales de la implementación.	23
6.2. Contrato operativo del código Python generado por el LLM.	25
6.3. Comprobaciones principales del validador de seguridad.	26
8.1. Niveles cualitativos de profundidad del análisis.	34
A.1. Batería cualitativa de consultas por nivel de profundidad.	45
A.2. Plantilla de revisión manual cualitativa.	47
A.3. Relación entre documentación Markdown y secciones de la memoria. . .	48

Índice de figuras

1.1. Mapa conceptual de relaciones entre inteligencia artificial, aprendizaje automático, redes neuronales, aprendizaje profundo e inteligencia artificial generativa. Fuente: Analytics Vidhya [1].	2
5.1. Arquitectura del flujo LLM con generación de código y ejecución controlada.	20

Capítulo 1

Introducción

1.1. Contexto y definición del problema

El análisis financiero histórico combina carga de datos, validación de entradas, selección del tipo de análisis, cálculo de métricas, generación de código, interpretación de resultados y comunicación final al usuario. En muchos prototipos basados en modelos de lenguaje estas fases se mezclan dentro de una única llamada generativa, lo que dificulta la trazabilidad del resultado y aumenta el riesgo de errores silenciosos.

Este trabajo plantea una aproximación controlada: el sistema parte de una entrada estructurada con consulta, tickers, fechas y rutas CSV; valida los datos mínimos; utiliza un LLM central para interpretar la intención; genera código Python; valida el contrato de seguridad del script; ejecuta el código en un entorno controlado; y finalmente usa el LLM para explicar las métricas obtenidas.

Para situar el alcance tecnológico del trabajo, la Figura 1.1 resume la relación entre inteligencia artificial, aprendizaje automático, aprendizaje profundo e inteligencia artificial generativa. El TFM no se centra en entrenar modelos ni en desarrollar nuevas arquitecturas neuronales, sino en integrar modelos de lenguaje dentro de un sistema trazable de análisis financiero histórico.

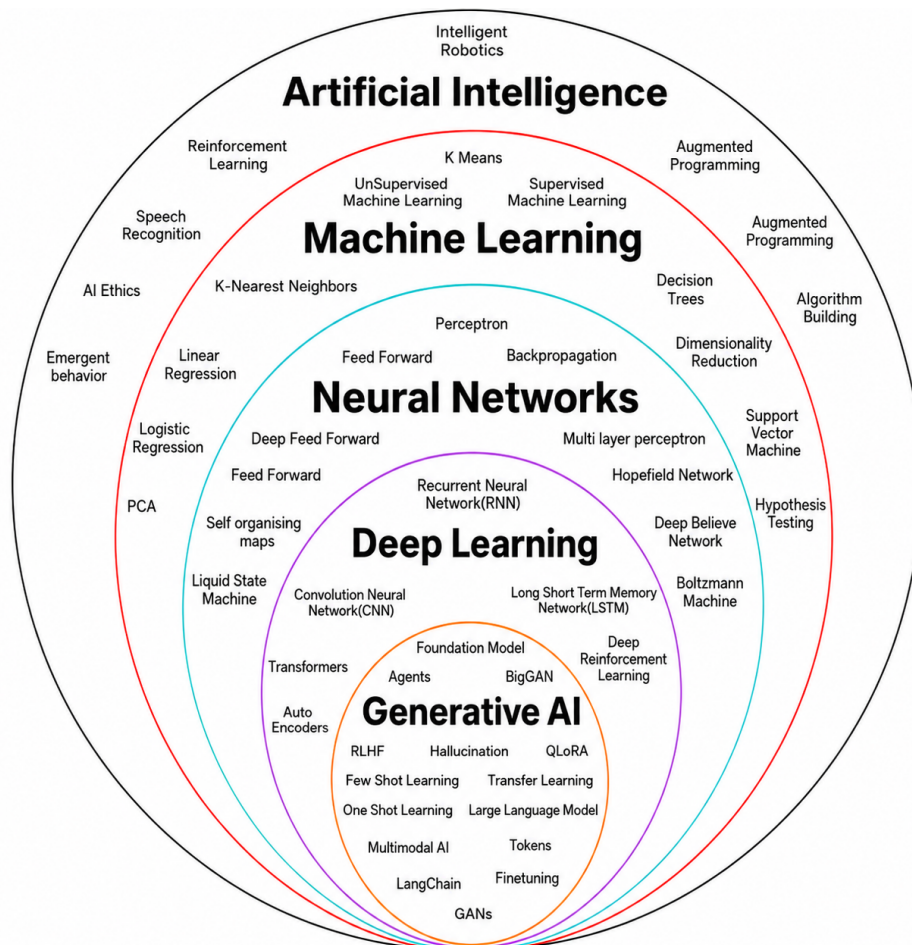


Figura 1.1: Mapa conceptual de relaciones entre inteligencia artificial, aprendizaje automático, redes neuronales, aprendizaje profundo e inteligencia artificial generativa. Fuente: Analytics Vidhya [1].

1.2. Motivación

La motivación principal del proyecto es explorar cómo un flujo LLM con ejecución controlada puede aportar orden, modularidad y robustez a un sistema de análisis financiero asistido por inteligencia artificial. La aplicación de modelos de lenguaje al dominio financiero resulta atractiva por su capacidad para interpretar peticiones, generar código y adaptar el lenguaje al usuario, pero también exige mecanismos de control, especialmente cuando la respuesta final puede confundirse con una recomendación financiera.

El interés del trabajo no está en construir un producto financiero completo, sino en estudiar si un prototipo incremental puede convertir consultas históricas en cálculos trazables y explicaciones prudentes. Por ello se evita que el LLM actúe como una caja negra que responde directamente al usuario: el modelo interpreta la intencionalidad de la consulta y genera código, pero el sistema valida dicho código antes de ejecutarlo y

captura los resultados en formato estructurado. Después, el LLM redacta la respuesta final a partir de métricas ya calculadas. Si el proveedor no está configurado o falla, el sistema informa el error y no genera una respuesta alternativa sin modelo.

En esta memoria se utiliza el término MVP como *prototipo mínimo viable* dentro de una evolución por etapas, no como producto comercial cerrado. El primer MVP validó dos intencionalidades básicas, crecimiento y comparación; el segundo amplió la cobertura a seis tipos de análisis; y la versión actual incorpora generación de código controlada, validación de seguridad, ejecución aislada, artefactos auditables y comparación entre proveedores LLM. Esta evolución se detalla en la Sección 4.2.

1.3. Objetivos

El objetivo general del trabajo es desarrollar y validar un prototipo basado en LLM capaz de ejecutar análisis financieros históricos a partir de entradas estructuradas y datos en CSV.

Los objetivos específicos son:

- Definir un flujo que separe validación, interpretación de intención, generación de código, control de seguridad, ejecución e interpretación final.
- Utilizar un LLM central para comprender la consulta e identificar la intencionalidad financiera.
- Generar código Python a partir de la intención interpretada y del mensaje original del usuario.
- Validar el código generado mediante contrato de script, importaciones permitidas y salida JSON.
- Ejecutar el código en un entorno controlado y conservar artefactos auditables.
- Redactar la respuesta final mediante LLM a partir de las métricas obtenidas.
- Evaluar el MVP mediante casos funcionales y escenarios de error controlado.

1.4. Alcance

El sistema se limita al análisis histórico-descriptivo de activos financieros a partir de datos ya descargados. No realiza predicción futura, recomendación de inversión ni análisis fundamental complejo. Esta restricción es deliberada: el foco del TFM no es construir un asesor financiero, sino estudiar una arquitectura LLM trazable para análisis histórico con generación y ejecución controlada de código.

1.5. Estructura de la memoria

La memoria se organiza siguiendo la lógica de desarrollo del proyecto. En primer lugar se presenta el estado de la cuestión, con especial atención a modelos de lenguaje, uso de herramientas externas y generación controlada de código. A continuación se describen los datos utilizados, su trazabilidad y su calidad. Después se expone la metodología seguida durante el desarrollo.

Los capítulos centrales describen el diseño del sistema, la implementación del MVP y la integración de modelos de lenguaje. Posteriormente se presentan los resultados de evaluación comparando proveedores LLM. Finalmente se discuten los aspectos éticos y de seguridad, se resumen las conclusiones y se plantean líneas de trabajo futuro.

Capítulo 2

Estado de la cuestión

2.1. Enfoque de la revisión

El objetivo de este capítulo es situar el TFM dentro de tres líneas de trabajo relacionadas: modelos de lenguaje aplicados a finanzas, sistemas LLM con herramientas externas y generación controlada de código para análisis de datos. La revisión no pretende cubrir todo el campo de la inteligencia artificial financiera, sino identificar referencias que permitan comparar la propuesta desarrollada.

El sistema de este TFM no entrega directamente la primera respuesta generativa del modelo. El LLM interpreta la consulta y genera código, pero el sistema valida el script, lo ejecuta en un entorno controlado y vuelve a usar el modelo para explicar resultados ya calculados. Por ello, resultan especialmente relevantes los trabajos que combinan lenguaje natural, datos financieros, herramientas, generación de código y mecanismos de control.

2.2. Modelos de lenguaje en finanzas

La aplicación de procesamiento del lenguaje natural al dominio financiero cuenta con antecedentes previos a los modelos generativos actuales. FinBERT adapta modelos tipo BERT al análisis de sentimiento financiero y muestra la utilidad de especializar modelos lingüísticos en textos de mercado [2]. Con los modelos de lenguaje de gran escala, trabajos como BloombergGPT y FinGPT amplían el foco hacia modelos entrenados o adaptados con datos financieros [3], [4].

Estos trabajos se sitúan en una escala distinta a la de este TFM. Su foco principal es entrenar o adaptar modelos para finanzas, mientras que este proyecto no entrena un modelo propio. La aportación aquí es arquitectónica y metodológica: usar modelos disponibles para interpretar consultas, generar código y explicar resultados dentro de un flujo verificable.

2.3. LLM con herramientas y ejecución

En el plano técnico, el diseño del sistema se relaciona con la literatura sobre modelos capaces de razonar y actuar con herramientas externas. ReAct introdujo la combinación de razonamiento y actuación [5]. Toolformer mostró que los modelos pueden aprender a decidir cuándo llamar a herramientas externas [6]. Las revisiones sobre agentes autónomos basados en LLM describen esta tendencia como un paso desde modelos conversacionales hacia sistemas capaces de planificar, actuar y observar resultados en entornos externos [7].

El TFM se inspira en esta separación entre razonamiento y uso de herramientas, pero fija de antemano los nodos del flujo: validación de entrada, análisis LLM, generación de código, control de seguridad, ejecución e interpretación. Esta decisión reduce flexibilidad frente a un agente autónomo, pero aumenta control, depuración y reproducibilidad.

2.4. Generación de código

La generación automática de código es una de las capacidades más relevantes de los LLM. El trabajo sobre Codex y HumanEval mostró que los modelos entrenados con código pueden sintetizar programas a partir de descripciones en lenguaje natural [8]. En ciencia de datos, DS-1000 propuso un banco de prueba específico para generación de código en tareas realistas con librerías habituales de análisis de datos [9].

Estos trabajos justifican que la generación de código sea una línea natural para sistemas de análisis financiero asistidos por modelos de lenguaje. También muestran un riesgo importante: el código generado debe validarse antes de ejecutarse. En el presente TFM, la generación de código mediante LLM forma parte del flujo principal, pero se somete a un control de seguridad que comprueba contrato del script, importaciones permitidas y salida JSON.

2.5. Fiabilidad y trazabilidad

Una limitación conocida de los modelos generativos es la posibilidad de producir respuestas plausibles pero no fieles a los datos [10]. En tareas de conocimiento, la generación aumentada con recuperación propuso reducir este problema mediante apoyo explícito en fuentes externas [11]. Aunque este TFM no implementa un sistema RAG documental, adopta una lógica de anclaje similar: el modelo trabaja con una entrada estructurada, CSV locales y resultados producidos por ejecución controlada.

En análisis financiero, una respuesta natural pero no trazable tiene poco valor académico. El sistema debe poder explicar qué datos se han usado, qué código se ha

ejecutado y qué limitaciones tiene la interpretación. Por ello, la memoria separa datos, interpretación de intención, generación de código, validación, ejecución e interpretación final.

2.6. Comparativa con trabajos relacionados

Trabajo	Dominio	Enfoque	Relación con este TFM
FinBERT [2]	PLN financiero	Modelo especializado para sentimiento financiero.	Muestra la utilidad de adaptar lenguaje al dominio financiero, pero no aborda generación controlada de código.
BloombergGPT [3]	LLM financiero	Modelo grande entrenado con datos financieros y generales.	Se centra en entrenamiento y evaluación de un modelo financiero; este TFM integra modelos existentes sin entrenarlos.
FinGPT [4]	LLM financiero abierto	Recursos abiertos y flujo de datos para modelos financieros.	Comparte la preocupación por datos y apertura, pero el TFM se enfoca en una arquitectura de análisis histórico con código generado.
ReAct y Toolformer [5], [6]	LLM con herramientas	Razonamiento combinado con acciones y llamadas a herramientas.	Inspiran la separación entre razonamiento, herramientas y observación de resultados.
Codex y DS-1000 [8], [9]	Generación de código	Síntesis y evaluación de código funcional o de ciencia de datos.	Justifican la generación de código del MVP y la necesidad de validaciones estrictas.

Tabla 2.1: Comparativa entre trabajos relacionados y la propuesta del TFM.

2.7. Posicionamiento de la propuesta

La aportación principal no es entrenar un nuevo modelo financiero ni construir un asesor de inversión. La propuesta consiste en diseñar y validar un flujo LLM para análisis financiero histórico en el que el modelo interpreta la consulta, genera código y redacta la respuesta final, mientras el sistema mantiene validación, ejecución controlada y trazabilidad sobre los artefactos.

Capítulo 3

Datos y material de trabajo

3.1. Fuente de datos y datos obtenidos

El sistema trabaja con datos financieros históricos descargados previamente mediante `yfinance` [12] y almacenados en ficheros CSV. Yahoo Finance publica información de mercado, fichas de activos y series históricas para acciones, fondos cotizados, índices, divisas, criptomonedas y materias primas. La librería `yfinance` permite acceder de forma programática a parte de esta información disponible en Yahoo Finance [13], incluyendo campos habituales como apertura, máximo, mínimo, cierre, cierre ajustado y volumen. En este trabajo se utiliza como herramienta práctica de descarga para un contexto académico, no como proveedor garantizado para producción.

Cada CSV se conserva junto a un fichero de metadatos en formato JSON. Este metadato registra la consulta original, la intención analítica, los símbolos financieros implicados, el intervalo temporal y los parámetros de descarga. La decisión permite separar la fase de obtención de datos de la fase de análisis: el MVP no depende de que la API externa esté disponible durante cada ejecución, sino que opera sobre un conjunto de datos congelado y revisable.

La Tabla 3.1 muestra los datos obtenidos y utilizados como base del TFM. No se incluyen todos los nombres de fichero en la tabla para evitar una presentación demasiado técnica; los nombres completos se mantienen en `data/raw` y en el resumen procesado `data/processed/eda_resumen_ficheros.csv`.

Caso de datos	Activos	Cobertura	Uso en el MVP
NVDA cinco años	NVDA	2021–2026, diario	Crecimiento de precio y validación de respuestas de Nivel A.
NVDA frente a AMD	NVDA, AMD	2024–2026, diario	Comparación entre activos y cálculo de rentabilidad relativa.
AAPL tres meses	AAPL	2025–2026, diario	Visión descriptiva de un activo y análisis técnico básico.
QQQ frente a SPY	QQQ, SPY	2024, diario	Retorno, riesgo, drawdown y visualizaciones comparativas.
S&P 500 desde 2020	<code>^GSPC</code>	2020–2026, diario	Serie amplia de mercado para pruebas de histórico largo.
Bitcoin 2024	BTC-USD	2024, diario	Activo cripto con alta variabilidad.
EUR/USD horario	EURUSD=X	10 días, 1h	Ejemplo de divisa con frecuencia intradía.
Oro horario	GC=F	1 semana, 1h	Materia prima con frecuencia intradía.

Tabla 3.1: Datos obtenidos y finalidad de cada conjunto dentro del MVP.

La variedad de activos permite comprobar que el sistema no está limitado a una acción concreta. También fuerza al flujo LLM a trabajar con horizontes distintos, desde consultas diarias de varios años hasta series horarias de pocos días.

3.2. Catálogo de consultas y proceso de descarga

El punto de partida del conjunto de datos es el catálogo `data/catalog/query_cases.json`. Cada entrada define la consulta de referencia, la intención esperada, los tickers, el rango temporal o período relativo, el intervalo de observación y los posibles avisos. Este catálogo actúa como contrato entre la fase de preparación de datos y el flujo de análisis.

El proceso utilizado para construir el conjunto de datos puede resumirse en cuatro pasos:

1. Definir o revisar la consulta en el catálogo, por ejemplo una comparativa entre QQQ y SPY o una descarga de NVDA a cinco años.
2. Transformar esa consulta en parámetros de descarga: lista de tickers, `start`, `end`, `period` e `interval`.

3. Descargar las series con `yfinance` y guardar el resultado en `data/raw` como CSV, junto a un `.metadata.json` con los parámetros usados.
4. Ejecutar `scripts/generate_data_eda.py` para normalizar los CSV, generar tablas procesadas y producir el informe exploratorio auxiliar.

En el repositorio actual, los CSV crudos ya están congelados. Por tanto, para reproducir la parte documentada de esta memoria no es necesario volver a llamar a Yahoo Finance: basta con conservar `data/raw` y ejecutar el script de EDA si se quieren regenerar las tablas procesadas. Si se quisiera ampliar el conjunto de datos, el procedimiento correcto sería añadir primero la consulta al catálogo y después generar un nuevo CSV con su metadato asociado.

Cada consulta se transforma en una representación estructurada antes de entrar en el flujo LLM. El siguiente bloque muestra un ejemplo simplificado:

```
{
  "query": "Compárame QQQ y SPY en 2024 de forma clara,
           con tabla y gráfica normalizada",
  "tickers": ["QQQ", "SPY"],
  "csv_paths": [
    "data/raw/qqq_y_spy_desde_20240101_hasta_20241231.csv"
  ],
  "intent": "compare_assets",
  "start": "2024-01-01",
  "end": "2024-12-31",
  "period": null,
  "interval": "1d",
  "warnings": []
}
```

La Tabla 3.2 resume el significado de los campos principales.

Elemento	Descripción	Ejemplo
Consulta original	Texto introducido o definido como caso de prueba.	Comparar Nvidia y AMD.
Intención analítica	Tipo de análisis que debe ejecutarse.	Comparación de activos.
Símbolos financieros	Activos sobre los que se realiza el análisis.	NVDA, AMD.
Rango temporal	Período relativo o fechas de inicio y fin.	Dos años.
Intervalo	Frecuencia de los datos históricos.	Diario.
Avisos	Observaciones detectadas al preparar la entrada.	Sin avisos.

Tabla 3.2: Estructura semántica de una consulta financiera antes de su procesamiento.

3.3. Estructura de los CSV

Los ficheros generados por `yfinance` contienen columnas de tipo OHLCV: apertura, máximo, mínimo, cierre, cierre ajustado y volumen. Cuando hay más de un ticker, las columnas se guardan con cabeceras multinivel: el primer nivel identifica el símbolo financiero y el segundo nivel identifica el campo de mercado. Por este motivo, los CSV no se leen como tablas planas con una sola fila de cabecera, sino con `header=[0, 1]`.

La Tabla 3.3 resume los campos usados por el sistema. La columna de cierre es la variable principal, porque de ella dependen las métricas de crecimiento, rentabilidad, volatilidad, drawdown e indicadores técnicos básicos.

Campo	Uso principal	Observación
Open	Precio de apertura.	Se conserva como contexto, aunque no es obligatorio en todas las respuestas.
High y Low	Rango máximo y mínimo.	Útiles para revisar amplitud diaria o intradía.
Close	Cálculo financiero central.	Base de rentabilidad, volatilidad, drawdown y medias móviles.
Adj Close	Precio ajustado.	Se conserva si aparece en la descarga original.
Volume	Actividad negociada.	Puede ser cero o menos informativo en divisas y algunos futuros.

Tabla 3.3: Campos OHLCV presentes en los CSV de mercado.

La estructura multinivel también explica por qué el proyecto incluye una capa de

normalización. El script de EDA transforma cada combinación caso-ticker en formato largo, con columnas explícitas para `case_id`, `source_file`, `ticker`, `interval`, fecha y campos OHLCV. Este formato facilita auditar métricas por serie y evita que el código generado por el LLM tenga que resolver por sí solo todas las variantes de cabecera.

3.4. Análisis exploratorio de datos

El análisis exploratorio se realizó con `scripts/generate_data_eda.py`. Este script lee los CSV descargados con `yfinance`, interpreta las cabeceras multinivel, normaliza los datos a formato largo y genera tres salidas procesadas: `eda_dataset_largo.csv`, `eda_resumen_ficheros.csv` y `eda_resumen_series.csv`. Además, produce un informe textual en `docs/eda_datos_tfm.txt`.

El análisis confirma que el conjunto contiene 5.080 filas normalizadas, con un rango observado entre el 2 de enero de 2020 y el 17 de marzo de 2026. Los intervalos considerados son diario (1d) e intradía horario (1h).

Indicador	Resultado
Casos definidos en catálogo	10
CSV analizados	8
Series por símbolo y caso	10
Filas normalizadas	5.080
Nulos en precios de cierre	0
Duplicados por fecha	0

Tabla 3.4: Resumen de cobertura y calidad del conjunto de datos.

Caso	Tickers	Filas	Rango observado
AAPL tres meses	AAPL	60	2025-12-17 a 2026-03-16
NVDA frente a AMD	AMD, NVDA	500	2024-03-18 a 2026-03-16
NVDA cinco años	NVDA	1.255	2021-03-17 a 2026-03-16
Bitcoin 2024	BTC-USD	365	2024-01-01 a 2024-12-30
S&P 500 desde 2020	^GSPC	1.558	2020-01-02 a 2026-03-16
QQQ frente a SPY	QQQ, SPY	251	2024-01-02 a 2024-12-30

Tabla 3.5: Muestra de cobertura temporal obtenida en el EDA.

Además del resumen agregado, se calcularon métricas descriptivas por serie. La Tabla 3.6 muestra una selección de resultados. Se observan activos con crecimientos muy elevados, como NVDA en cinco años, series con caídas acumuladas significativas,

como AMD en la comparativa con NVDA, y activos de menor volatilidad relativa, como SPY. Esta diversidad obliga al sistema a redactar respuestas para escenarios con magnitudes y niveles de riesgo distintos.

Serie	Retorno total	Volatilidad anual	Máx. caída
NVDA, 5 años	1273.33 %	51.31 %	-66.36 %
BTC-USD, 2024	109.76 %	44.13 %	-26.18 %
NVDA, 2 años	107.13 %	49.42 %	-36.89 %
S&P 500, desde 2020	105.64 %	20.77 %	-33.93 %
QQQ, 2024	28.07 %	17.99 %	-13.56 %
SPY, 2024	24.45 %	12.63 %	-8.41 %
AMD, 2 años	3.11 %	55.10 %	-58.98 %
AAPL, 3 meses	-7.00 %	24.15 %	-10.07 %

Tabla 3.6: Métricas exploratorias por serie utilizadas para contextualizar los casos de prueba.

3.5. Calidad, cobertura y limitaciones

La ausencia de nulos en el precio de cierre es relevante porque las métricas principales del MVP dependen directamente de esta columna. También se comprobó la ausencia de duplicados por fecha en los ficheros analizados. Estas condiciones permiten ejecutar las intenciones actuales sin introducir imputaciones adicionales.

Como limitación, las descargas basadas en períodos relativos, por ejemplo tres meses o cinco años, dependen de la fecha en la que se generó el CSV. Para una versión final completamente cerrada sería recomendable fijar fechas explícitas de inicio y fin en todos los casos. También debe considerarse que los datos proceden de Yahoo Finance mediante `yfinance`. Al tratarse de una herramienta abierta no afiliada ni avalada por Yahoo, su uso se considera adecuado para un contexto académico y experimental, pero en un entorno productivo habría que estudiar licencias, disponibilidad, estabilidad del proveedor y derechos de uso de los datos descargados [12].

3.6. Trazabilidad documental y artefactos reproducibles

La conservación de notebooks, informes Markdown y tablas procesadas es profesional siempre que se presente como trazabilidad metodológica y no como material informal. En este trabajo se mantienen como evidencias auxiliares para reconstruir

decisiones de diseño, parámetros de descarga, estructura de los CSV y resultados intermedios.

Los cuadernos auxiliares tienen funciones diferenciadas. El primero se utilizó como laboratorio para probar nuevos símbolos financieros, períodos e intervalos antes de incorporarlos al conjunto definitivo. El segundo documenta la relación entre consultas de referencia y CSV esperados. El tercero funciona como referencia principal del análisis exploratorio formal, apoyado por `scripts/generate_data_eda.py` y por las tablas agregadas usadas en esta memoria.

También se revisaron los documentos Markdown generados durante la evolución del proyecto. El documento de arquitectura sirvió para consolidar la descripción del flujo LLM con generación de código controlada; los documentos de evaluación cualitativa y amplitud se emplearon para definir los niveles A/B/C; y las salidas de ejecución se reservaron como evidencias de apoyo para el capítulo de resultados y los anexos. Esta separación evita mezclar notas de desarrollo con resultados finales, pero mantiene la trazabilidad de las decisiones metodológicas.

Capítulo 4

Metodología

4.1. Enfoque de desarrollo

El desarrollo se ha reorganizado hacia un flujo más simple y trazable. El sistema separa la interpretación de la consulta, la generación y ejecución controlada del código y la explicación del resultado.

La metodología sigue una lógica incremental:

1. Definir una entrada estructurada con consulta, tickers, fechas y CSV.
2. Validar campos mínimos antes de llamar al modelo.
3. Usar un LLM central para interpretar la intencionalidad financiera.
4. Generar código Python a partir de la interpretación y del mensaje original.
5. Validar el código antes de ejecutarlo.
6. Ejecutar el script en un entorno controlado y guardar artefactos.
7. Usar el LLM para explicar las métricas obtenidas.

4.2. Evolución incremental del MVP

El desarrollo del prototipo no se planteó como una implementación cerrada desde el inicio, sino como una sucesión de versiones del MVP. La primera versión tuvo como objetivo validar que el flujo completo podía funcionar de extremo a extremo. Para ello se trabajó con un LLM de OpenAI o compatible con OpenAI y con dos intencionalidades básicas: crecimiento de un activo y comparación entre activos. Esta fase permitió comprobar la viabilidad inicial de la cadena formada por consulta, interpretación, cálculo y respuesta.

Una vez validado ese flujo mínimo, el segundo MVP amplió la cobertura funcional de dos a seis intencionalidades analíticas: crecimiento de precio, comparación de activos, visión descriptiva de un activo, análisis de retornos, análisis de riesgo histórico y análisis técnico básico. Esta ampliación obligó a revisar contratos de entrada y salida, estructura de los planes analíticos, gestión de datos y formato de las respuestas.

Posteriormente, el proyecto evolucionó hacia la arquitectura actual, centrada en LLM y generación de código controlada. En esta etapa se introdujeron validación de seguridad, ejecución aislada del script generado, guardado de artefactos, reintentos ante errores LLM y reparación del código cuando falla la validación o la ejecución. Finalmente, el sistema se preparó para comparar varios modelos y para iniciar una fase de consultas más complejas sobre los mismos datos históricos.

4.3. Criterios de validación

La validación del prototipo se apoya en tres niveles. El primero comprueba entradas mínimas: consulta no vacía, tickers, rutas CSV existentes y fechas en formato correcto. El segundo revisa que el código generado cumpla un contrato básico: función `main()`, uso de `json.dumps`, importaciones permitidas y salida JSON. El tercero comprueba que la respuesta final evita predicciones, recomendaciones de inversión y datos externos no presentes en la ejecución.

4.4. Criterios cualitativos de calidad del análisis

Además de la validación técnica, la metodología incorpora una revisión humana de la calidad del análisis generado. Esta revisión es necesaria porque dos ejecuciones pueden terminar con estado `completed` y, aun así, ofrecer niveles de utilidad distintos para el usuario. El objetivo no es obtener una puntuación matemática absoluta, sino valorar si la respuesta es completa, prudente, trazable y coherente con la consulta.

Se considera buen análisis aquel que responde directamente a la petición, calcula o muestra los datos esenciales, respeta el formato solicitado por el usuario y separa con claridad dato histórico, interpretación y limitaciones. Esta definición se apoya en recomendaciones de educación financiera que advierten que el rendimiento histórico debe presentarse indicando cómo se calcula y sin tratarlo como garantía de resultados futuros [14]. La selección de métricas parte de la relación entre riesgo y retorno [15] y de medidas habituales como rentabilidad, volatilidad y correlación en series de retornos históricos [16]. El drawdown se incluye como medida interpretable de caídas históricas porque permite observar pérdidas desde máximos previos usando datos observados [17].

Estas fuentes se usaron como guía para traducir una petición en lenguaje natural a una salida analítica razonable: primero se identifica si el usuario pide rentabilidad,

comparación, riesgo, evolución temporal o explicación; después se seleccionan métricas coherentes con esa intención; y finalmente se exige que la interpretación distinga entre resultados históricos observados, lectura descriptiva y límites del análisis. De este modo, la elección de “buen análisis” no queda basada solo en preferencia subjetiva, sino en criterios documentados de presentación de rendimiento, relación riesgo-retorno y visualización comprensible.

La revisión diferencia dos responsabilidades. El primer agente no debe redactar una respuesta financiera final: su tarea es interpretar la consulta y generar código que produzca métricas, tablas o datos de visualización suficientes para responderla. El segundo agente no debe inventar datos ni completar huecos con conocimiento externo: su tarea es transformar la salida estructurada de la ejecución en una explicación comprensible, separando dato histórico, interpretación y limitaciones. De este modo, la calidad final depende de dos eslabones evaluables por separado: la calidad del material generado por el primer agente y la calidad de la explicación redactada por el segundo.

4.5. Niveles de profundidad del análisis

La profundidad del análisis se adapta a lo que solicita el usuario. Se definen tres niveles:

- **Nivel A, análisis simple por defecto:** se aplica cuando la consulta es sencilla y no pide formato especial. La salida esperada es un texto breve y una tabla o bloque estructurado con métricas básicas. No se fuerza la generación de gráficas si la respuesta puede entenderse con datos resumidos.
- **Nivel B, análisis mejorado:** se aplica cuando el usuario pide un análisis más completo, una comparativa clara o una explicación con más detalle. La salida esperada combina texto, tabla de métricas y una gráfica principal o los datos necesarios para representarla.
- **Nivel C, análisis profesional o enriquecido:** se aplica cuando el usuario pide expresamente un informe detallado, profesional, avanzado o con varias visualizaciones concretas. Puede incluir varias tablas, evolución normalizada, drawdown, retornos, volatilidad, correlación, volumen, medias móviles o desglose por períodos.

Esta graduación evita sobrecargar consultas simples y, al mismo tiempo, permite evaluar si el sistema es capaz de aumentar la riqueza del análisis cuando el usuario lo solicita. En series temporales financieras, los gráficos de línea son adecuados para mostrar evolución en el tiempo; en comparaciones entre activos con escalas distintas se

prioriza la normalización, y se evita abusar de demasiadas series o ejes dobles para no inducir lecturas confusas [18].

En las consultas de Nivel B y C, el primer agente puede producir datos extensos para visualizaciones. Para evitar que esas series completas saturen el contexto del segundo agente, el workflow conserva la salida íntegra como artefacto auditable y envía al LLM interpretador una versión compactada. Esta decisión mantiene la trazabilidad técnica y, a la vez, reduce el riesgo de errores por exceso de contexto.

4.6. Evaluación con proveedores LLM

La evaluación se ha preparado para comparar tres configuraciones compatibles con OpenAI: Groq con `llama-3.3-70b-versatile` [19], modelo Llama 3.3 70B Instruct orientado a razonamiento general y baja latencia; Gemini con `gemini-2.5-flash` [20], modelo Flash de Gemini 2.5 orientado a rapidez y razonamiento; y `openai/gpt-oss-20b` [21] servido mediante vLLM en el entorno universitario, modelo abierto de la familia gpt-oss usado por su capacidad de razonamiento, seguimiento de instrucciones y generación de código. El proceso de depuración principal se documentó con `openai/gpt-oss-20b` [21]; posteriormente se comprobó la misma batería compleja con Groq usando `llama-3.3-70b-versatile` [19] y se realizaron pruebas puntuales con Gemini usando `gemini-2.5-flash` [20]. En todos los casos el flujo conceptual es el mismo: análisis de la consulta, generación de código, control de seguridad, ejecución e interpretación final. Las diferencias entre proveedores se observan en latencia, estabilidad, calidad textual y cumplimiento del formato esperado.

4.7. Revisión humana

El flujo no se plantea como una automatización ciega. La revisión humana posterior forma parte del proceso metodológico: permite comprobar si la interpretación inicial, el código generado, los resultados estructurados y la respuesta final son coherentes con la consulta del usuario y con los datos históricos utilizados.

La plantilla de revisión manual considera, como mínimo, comprensión de la consulta, selección de métricas, adecuación del código generado, cumplimiento de tablas o gráficas solicitadas, exactitud respecto a la salida ejecutada, claridad de la respuesta final, utilidad para el usuario, reconocimiento de limitaciones, ausencia de recomendaciones de inversión y trazabilidad entre consulta, código, métricas e interpretación. La valoración se expresa mediante categorías cualitativas, por ejemplo excelente, correcto, parcial o deficiente, para reconocer la subjetividad controlada de este tipo de evaluación.

Capítulo 5

Diseño del sistema

5.1. Arquitectura general

El sistema se ha diseñado como un workflow modular que transforma una consulta financiera estructurada en una respuesta analítica basada en código generado por LLM y ejecutado de forma controlada. La arquitectura separa de forma explícita las fases de validación, análisis LLM, generación de código, control de seguridad, ejecución e interpretación.

El estado compartido conserva la consulta normalizada, la interpretación estructurada, el código generado, la salida estándar, la salida de error, el código de retorno, los artefactos y la respuesta final. Esta estructura permite mantener trazabilidad entre la entrada inicial y el resultado entregado al usuario.

La separación entre agentes es una decisión central del diseño. El primer agente actúa como preparador de datos: entiende la petición, decide qué cálculos son necesarios y genera un script que produce resultados estructurados. El segundo agente actúa como intérprete: recibe resultados ya calculados y los convierte en una explicación para el usuario. Esta división evita que el sistema dependa solo de una respuesta textual del modelo y permite revisar por separado consulta, código, ejecución, métricas e interpretación.

La arquitectura se documentó también como artefacto interno de apoyo, recogido en el Anexo A.3. Ese documento sirvió para fijar el mapa conceptual del flujo, la responsabilidad de cada nodo y la decisión de tratar la generación de código como una etapa intermedia validable, no como una respuesta final. La memoria recoge esa lógica de forma resumida y la formaliza en la Figura 5.1.

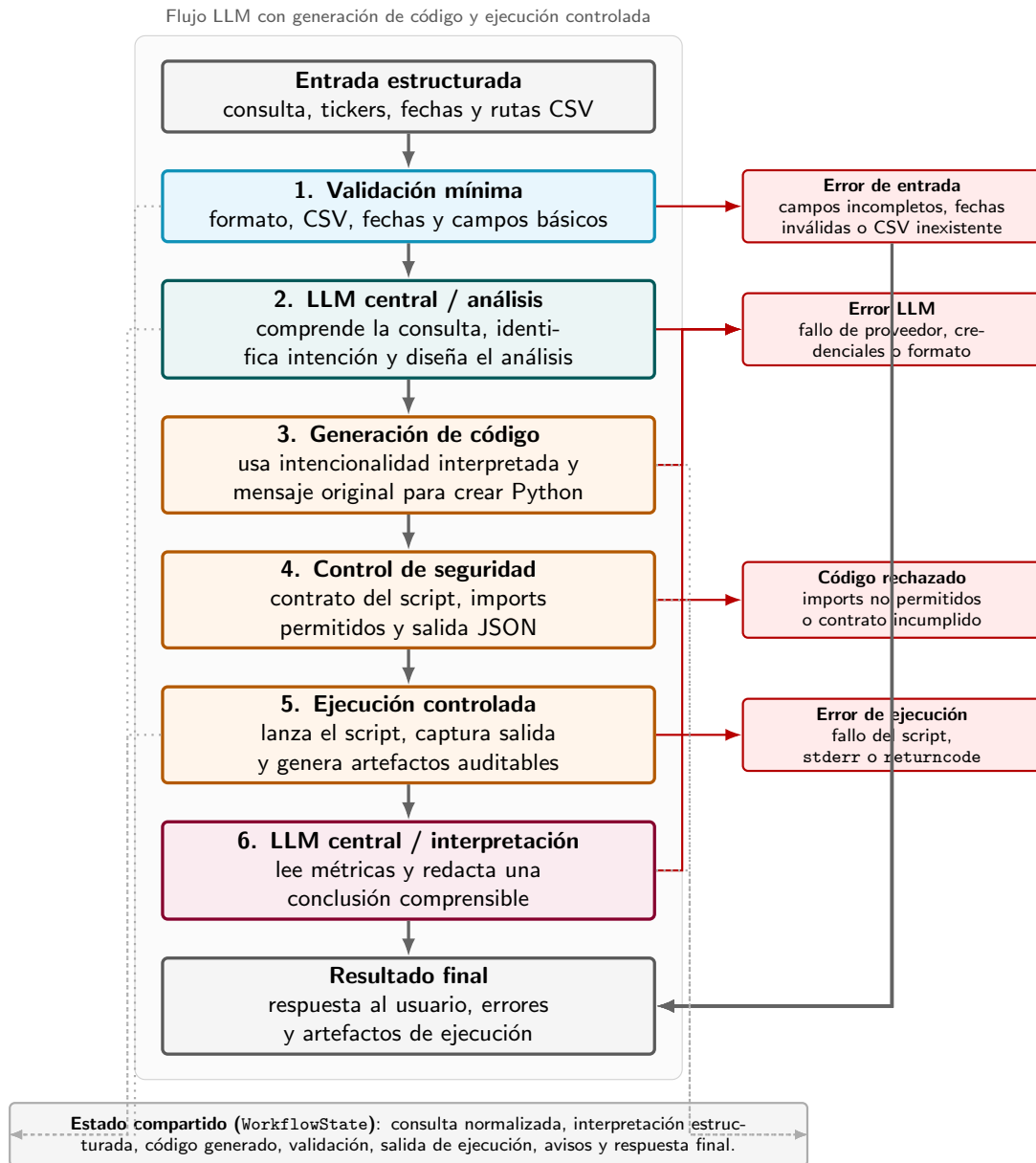


Figura 5.1: Arquitectura del flujo LLM con generación de código y ejecución controlada.

5.2. Nodos del flujo

El flujo se compone de los siguientes nodos:

- **Validación mínima:** comprueba la consulta, los tickers, las fechas y la existencia de CSV.
- **LLM central / análisis:** interpreta la consulta, identifica la intencionalidad financiera y define qué debe calcularse.

- **Generación de código:** recibe la interpretación y el mensaje original del usuario para producir un script Python.
- **Control de seguridad:** valida contrato del script, importaciones permitidas y salida JSON.
- **Ejecución controlada:** lanza el script en un proceso separado, captura stdout, stderr y artefactos.
- **LLM central / interpretación:** lee métricas y redacta una conclusión comprensible para el usuario.

La documentación interna de arquitectura ayudó a mantener una correspondencia directa entre estos nodos y los módulos reales del repositorio. En particular, el flujo separa los nodos que dependen del proveedor LLM de los nodos deterministas de validación, seguridad y ejecución. Esta separación facilita cambiar de modelo sin modificar la lógica principal del workflow y permite analizar los fallos por origen.

5.3. Contrato del código generado

El contrato del código generado es la interfaz entre la parte generativa del sistema y la parte determinista. Su función es convertir una salida libre del LLM en un artefacto que pueda revisarse, ejecutarse y comparar de forma sistemática. Sin este contrato, cada script podría tener una estructura distinta, imprimir texto no parseable, depender de recursos externos o mezclar cálculo con explicación, lo que haría difícil auditar el resultado.

El código generado no debe redactar la respuesta final. Su responsabilidad es calcular métricas y devolver una salida estructurada. El contrato mínimo exige una función `main()`, lectura de los datos de entrada recibidos por `argv[1]`, escritura de un único JSON por `stdout` y uso exclusivo de importaciones permitidas. Estas reglas permiten que el sistema trate todos los scripts generados de la misma forma: primero los valida, después los ejecuta en un proceso separado y finalmente interpreta su salida JSON.

Por tanto, el primer agente no se evalúa por lo convincente que suene una explicación, sino por si prepara datos suficientes para la consulta. Del mismo modo, el segundo agente no se evalúa por calcular nuevas métricas, sino por interpretar fielmente lo que el script ha devuelto. La respuesta final solo se considera sólida cuando ambos componentes funcionan de forma coherente.

El sentido práctico de este contrato es reducir el riesgo de tres problemas habituales en sistemas LLM con generación de código: ejecución de instrucciones no deseadas, respuestas imposibles de procesar automáticamente y pérdida de trazabilidad entre consulta, código, métricas y explicación final. Por ello, el contrato no es solo una

restricción técnica, sino una decisión metodológica: obliga a que el modelo produzca cálculo verificable antes de producir lenguaje natural.

5.4. Gestión de errores

Los errores se separan por origen: entrada inválida, error LLM, código rechazado, error de ejecución y error de interpretación. Esta separación facilita depuración y revisión humana posterior, porque permite saber si el problema procede de los datos, del proveedor LLM, del código generado o del entorno de ejecución.

Capítulo 6

Implementación

6.1. Estructura del código

La implementación se organiza en módulos pequeños:

Módulo	Responsabilidad
<code>schemas.py</code>	Define la entrada, el plan interpretado, los artefactos y el estado del workflow.
<code>validation.py</code>	Valida query, tickers, fechas y rutas CSV antes de llamar al LLM.
<code>nodes.py</code>	Implementa los nodos del flujo: análisis LLM, codegen, seguridad, ejecución e interpretación.
<code>src/llm/</code>	Contiene prompts, cliente compatible con OpenAI y funciones del pipeline LLM.
<code>code_security.py</code>	Revisa el código generado antes de ejecutarlo.
<code>code_runner.py</code>	Guarda el script, lanza la ejecución y captura stdout, stderr y artefactos.
<code>market_data.py</code>	Proporciona funciones auxiliares para leer CSV financieros descargados con <code>yfinance</code> .
<code>tests/</code>	Comprueba el workflow con llamadas LLM simuladas y escenarios de error.

Tabla 6.1: Módulos principales de la implementación.

6.2. Repositorio y organización del trabajo

El desarrollo se ha gestionado en un repositorio GitHub disponible en <https://github.com/CarlosRuiz4102/tfm>. El uso de control de versiones permite conservar la evolución del código, la memoria, los scripts de evaluación y la documentación auxiliar en un único espacio de trabajo. Esta organización facilita revisar qué parte corresponde a datos, qué parte corresponde al flujo LLM, qué parte corresponde a resultados y qué parte corresponde a documentación académica.

La estructura del repositorio separa responsabilidades de forma cercana a una práctica MLOps ligera:

- **data/**: contiene el catálogo de casos, los CSV históricos congelados y sus metadatos.
- **src/**: agrupa el código de aplicación, dividido en esquemas, grafo, integración LLM y ejecución controlada.
- **scripts/**: incluye utilidades para generar EDA, ejecutar evaluaciones y producir informes reproducibles.
- **tests/**: valida el workflow con respuestas LLM simuladas, evitando depender de APIs externas durante las pruebas básicas.
- **docs/**: conserva la memoria LaTeX, informes Markdown, figuras y registros de resultados.
- **results/**: almacena artefactos generados durante ejecuciones, como scripts, datos de entrada, registros y salidas JSON.

También se separan configuración y secretos. El fichero `.env` contiene claves locales y queda excluido del control de versiones, mientras que `.env.example` documenta las variables necesarias para reproducir la configuración sin exponer credenciales. Del mismo modo, `.gitignore` excluye entornos virtuales, cachés, registros y resultados generados de gran tamaño, manteniendo versionados los artefactos que aportan trazabilidad académica.

Aunque el proyecto no implementa un ciclo MLOps productivo completo con despliegue continuo, sí incorpora prácticas propias de ese enfoque: versionado del código, separación entre datos, código y resultados, scripts de evaluación repetibles, pruebas automatizadas, conservación de artefactos, documentación de configuraciones y registro explícito de fallos. Para el alcance del TFM, estas prácticas son suficientes para defender que el sistema no se limita a una demo puntual, sino que se ha organizado como un prototipo reproducible y auditable.

6.3. Contratos de datos

La entrada del sistema se representa mediante `FinancialQueryInput`. La interpretación generada por el LLM se almacena en `AnalysisPlan`, que contiene intencionalidad interpretada, tipo de análisis, métricas esperadas, requisitos de datos, requisitos de salida, preferencias de presentación y razonamiento resumido. El estado completo se conserva en `WorkflowState`.

La necesidad de funciones auxiliares de datos apareció durante las pruebas de desarrollo resumidas en `docs/planificacion_evaluacion_tfm.md`: los CSV generados por `yfinance` pueden contener cabeceras multinivel y formatos distintos según el activo o el intervalo. Para reducir errores repetidos en el código generado, se incorporaron funciones reutilizables de carga y normalización. Así, el LLM no tiene que resolver desde cero cada variante de lectura de CSV y puede concentrarse en el cálculo solicitado.

6.4. Pipeline LLM

El pipeline LLM se divide en tres llamadas principales. La primera interpreta la consulta y construye un plan estructurado. La segunda genera código Python usando tanto el plan como el mensaje original del usuario. La tercera interpreta la salida JSON producida por el script y redacta la respuesta final.

El prompt de generación de código exige que el script lea un payload por `argv[1]`, use librerías permitidas y escriba un único JSON en `stdout`. Esta restricción permite tratar el código generado como un artefacto ejecutable, auditable y validable.

El flujo completo puede resumirse como una cadena de transformación controlada:

prompt → código Python → validación de seguridad → ejecución →
JSON → interpretación final.

Esta cadena es importante porque el LLM no produce directamente una afirmación financiera final sin pasar antes por cálculos ejecutados. El primer bloque LLM prepara el análisis y genera el script; el entorno de ejecución produce datos estructurados; y el segundo bloque LLM redacta una explicación basada en esa salida.

Elemento	Contrato exigido
Entrada	El script recibe un único payload JSON por <code>argv[1]</code> , con consulta, tickers, fechas, intervalo y rutas CSV.
Lectura de datos	El código debe leer únicamente los CSV proporcionados por el workflow y no descargar datos externos.
Salida	Debe escribir un único objeto JSON en <code>stdout</code> , mediante <code>json.dumps</code> , para que el sistema pueda analizarlo.
Contenido	La salida debe incluir métricas, tablas o datos de visualización necesarios para responder a la consulta.
Artefactos	La ejecución conserva script, salida estándar, salida de error, código de retorno y metadatos de ejecución.
Errores	Si el script falla, el error queda registrado y el workflow puede devolver una explicación controlada o intentar reparación.

Tabla 6.2: Contrato operativo del código Python generado por el LLM.

6.5. Control de seguridad

Antes de ejecutar el script, el módulo `code_security.py` analiza el código con el AST de Python. El validador comprueba importaciones permitidas, llamadas bloqueadas como `eval` o `exec`, presencia de `main()` y uso de `json.dumps`. Si el código no cumple el contrato, el workflow termina con estado de error y no ejecuta el script.

El uso del árbol de sintaxis abstracta permite revisar la estructura del programa antes de ejecutarlo, en lugar de depender solo de búsquedas textuales. La validación comprueba nodos de importación, llamadas a funciones y elementos mínimos del contrato. Con ello se reduce el riesgo de que un script generado por LLM ejecute operaciones fuera del alcance del MVP. Esta capa no convierte el entorno en una sandbox completa, pero sí añade una barrera explícita entre generación y ejecución.

Comprobación	Finalidad
Imports permitidos	Limitar el código a librerías necesarias para análisis de datos y serialización.
Funciones bloqueadas	Evitar llamadas peligrosas o no necesarias, como <code>eval</code> y <code>exec</code> .
Función <code>main()</code>	Forzar una estructura ejecutable y sencilla de auditar.
Salida JSON	Garantizar que el resultado pueda pasar al nodo de interpretación sin ambigüedad.
Errores explícitos	Separar fallos de seguridad, fallos de ejecución y fallos de interpretación.

Tabla 6.3: Comprobaciones principales del validador de seguridad.

6.6. Reparación y compactación

Cuando el código generado no supera la validación o falla durante la ejecución, el workflow puede solicitar al LLM una reparación manteniendo la consulta original, el plan analítico y el detalle del error. La reparación no evita la validación: el código corregido vuelve a pasar por el mismo control de seguridad antes de ejecutarse. Si la reparación tampoco cumple el contrato, el caso se marca como error controlado.

En consultas de Nivel B o C, el script puede generar series visuales largas. Estas series son útiles como artefacto completo, pero pueden saturar el contexto del segundo agente. Por ello, antes de la interpretación final se compactan listas y textos extensos: se conserva una muestra representativa para el LLM interpretador y la salida completa queda guardada como evidencia auditable. Esta decisión permite mantener trazabilidad sin degradar la respuesta final por exceso de contexto.

Este ajuste quedó documentado en una nota interna de control de tamaño de sa-

lidas, referenciada en el Anexo A.3. La decisión práctica fue limitar o resumir datos de visualización enviados al segundo agente, manteniendo siempre la salida completa en los registros. De este modo, las consultas de Nivel B y C pueden producir material gráfico sin convertir la interpretación final en una llamada demasiado grande para el proveedor LLM.

6.7. Pruebas

Las pruebas automatizadas simulan las respuestas LLM para evitar dependencias externas durante su ejecución. De este modo se validan el workflow, el control de seguridad, la ejecución controlada y los errores de entrada sin consumir APIs reales.

Capítulo 7

Integración de modelos de lenguaje

7.1. Papel del LLM en el sistema

El trabajo incorpora modelos de lenguaje como parte central del flujo. El LLM no se limita a redactar la respuesta final: primero interpreta la consulta y construye una intencionalidad estructurada; después genera código Python a partir de dicha interpretación y del mensaje original; finalmente interpreta las métricas devueltas por la ejecución.

Esta organización permite aprovechar la flexibilidad del modelo sin ejecutar sus salidas de forma ciega. Entre generación y ejecución existe una capa de control de seguridad que revisa el script antes de lanzarlo.

7.2. Cliente compatible con OpenAI

La integración se ha preparado mediante un cliente compatible con APIs tipo OpenAI. Esta decisión permite alternar entre proveedores sin modificar el workflow principal. El sistema utiliza variables de entorno para seleccionar perfil, modelo, URL base y clave de API.

En esta memoria, cuando se comentan resultados ya ejecutados, se hace referencia al modelo `openai/gpt-oss-20b` [21] servido mediante vLLM en el entorno universitario. Este modelo abierto de la familia gpt-oss se usa por su capacidad de razonamiento, seguimiento de instrucciones y generación de código Python. El nombre `university` queda reservado como identificador técnico del perfil de configuración dentro del código, no como nombre académico del modelo.

La configuración operativa de los proveedores se dejó descrita en `docs/llm_api_setup.md`. Esa documentación distingue entre el identificador técnico del perfil, el proveedor utilizado y el modelo concreto, lo que evita mezclar resultados del workflow con detalles locales de configuración. Esta separación fue especialmente útil al repe-

tir evaluaciones con Groq, Gemini y el servidor vLLM universitario sin cambiar la arquitectura del sistema.

7.3. Configuraciones evaluadas

Se han preparado tres configuraciones principales:

- **Groq con llama-3.3-70b-versatile** [19]: proveedor externo compatible con OpenAI. El modelo corresponde a Llama 3.3 70B Instruct y se utiliza por su razonamiento general, seguimiento de instrucciones y baja latencia en las pruebas.
- **Gemini con gemini-2.5-flash** [20]: proveedor secundario para estudiar portabilidad y calidad comparativa. El modelo Flash de Gemini 2.5 se utiliza por su rapidez, capacidad de razonamiento y equilibrio entre latencia y calidad.
- **openai/gpt-oss-20b sobre vLLM universitario** [21]: modelo abierto de la familia gpt-oss ejecutado en el servidor compatible con OpenAI Chat Completions habilitado en el entorno universitario. Se usa como modelo principal por su capacidad de razonamiento, seguimiento de instrucciones y generación de código.

7.4. Gestión de errores de API

El diseño registra errores de red, cuota, indisponibilidad, credenciales o formato de respuesta. Si una llamada LLM falla, el sistema devuelve un error explícito para que la evaluación no mezcle resultados reales de modelo con respuestas programáticas.

La gestión de errores se concentra en el cliente compatible con OpenAI y en la capa de pipeline. El cliente encapsula las excepciones de la API en una excepción propia, `LLMClientError`, de modo que el resto del workflow no depende de detalles internos de cada proveedor. Esta abstracción permite tratar de forma homogénea fallos de conectividad, claves ausentes, límites de cuota, tiempos de espera agotados o respuestas vacías, aunque el origen técnico sea diferente en Groq, Gemini o el servidor vLLM universitario.

Además, no todos los errores LLM tienen el mismo significado dentro del sistema. En la fase de análisis, un fallo impide construir el plan inicial y el caso termina como error controlado. En la fase de generación de código, el error puede deberse a una respuesta no válida, a JSON mal formado o a un script que no cumple el contrato mínimo. En la fase de interpretación final, en cambio, puede existir una ejecución correcta y métricas calculadas, pero fallar la redacción final del modelo. Por este motivo el estado del workflow distingue entre peticiones inválidas, código rechazado, fallo de ejecución y casos `completed_with_error`.

Para reducir fallos por formato, el pipeline solicita JSON estricto en las fases de análisis y generación de código. Si la respuesta no puede parsearse, se realizan hasta tres intentos con un mensaje de corrección que pide devolver exclusivamente un objeto JSON válido. En la generación de código, además, se comprueba que el script contenga una función `main()` y que escriba un único JSON mediante `json.dumps`. Si el código generado no supera la validación de seguridad o falla durante la ejecución, el sistema puede pedir una reparación al LLM usando el error observado como contexto. Esta reparación queda registrada mediante avisos para no ocultar que el caso necesitó una segunda intervención del modelo.

Los prompts, el contrato de salida y el tratamiento de errores se revisaron a partir de ejecuciones reales documentadas en Markdown. En esas pruebas se observó que algunos fallos no se debían a una mala comprensión financiera, sino a detalles operativos: JSON incompleto, código no ejecutable, lectura incorrecta de CSV o salidas demasiado extensas. Por ello se combinaron reintentos de formato, reparación de código y compactación de salidas antes de la interpretación final.

En las evaluaciones reales se observaron varios tipos de fallo útiles para el análisis del TFM:

- **Errores de cuota:** por ejemplo, respuestas 429 cuando un proveedor supera el límite diario de uso. Estos casos no indican que el razonamiento del modelo sea incorrecto, sino que la comparación experimental queda condicionada por disponibilidad.
- **Errores de conexión:** pueden aparecer si la ejecución se lanza sin acceso de red real o si el proveedor no responde. En la documentación de resultados se marcan como diagnóstico de entorno y no como evidencia de rendimiento del modelo.
- **Errores de formato:** se producen cuando el modelo entiende la tarea, pero devuelve texto o código fuera del JSON esperado. Estos fallos justifican los reintentos y la validación posterior.
- **Errores por contexto o tamaño de salida:** algunas consultas visuales generan estructuras largas, por ejemplo series temporales completas. Para mitigarlo, antes de la interpretación final se compactan listas y textos extensos, manteniendo la salida completa como artefacto auditable.

Esta gestión de errores es importante metodológicamente. La memoria no presenta únicamente tasas de acierto, sino también las condiciones bajo las que falla el sistema. Separar errores de proveedor, errores de formato, errores de seguridad y errores de ejecución permite interpretar mejor los resultados y evita atribuir al modelo un problema que en realidad pertenece a la cuota, a la conectividad o al contrato técnico de salida.

7.5. Gestión de credenciales

Las claves de Groq con `llama-3.3-70b-versatile` [19], Gemini con `gemini-2.5-flash` [20] y del servidor vLLM universitario que sirve `openai/gpt-oss-20b` [21] se gestionan mediante variables de entorno en el fichero local `.env`. Este archivo no se versiona y no debe compartirse. El repositorio incluye configuraciones de ejemplo, pero una ejecución real con API solo es posible cuando el entorno local contiene una clave válida.

Capítulo 8

Resultados y evaluación

Este capítulo define el protocolo con el que se realizará la evaluación definitiva del prototipo. Las ejecuciones realizadas durante el desarrollo se utilizaron para depurar el sistema, pero no se presentan como resultados finales: la comparación definitiva debe realizarse con una batería fijada previamente, los mismos datos locales y criterios de revisión homogéneos.

8.1. Objetivo de la evaluación

El objetivo no es demostrar que el sistema prediga el mercado, sino comprobar si la arquitectura interpreta consultas financieras históricas, genera código Python adecuado, valida su seguridad, ejecuta métricas trazables y redacta conclusiones comprensibles.

El proyecto contempla tres perfiles de ejecución:

- Groq con `llama-3.3-70b-versatile` [19].
- Gemini con `gemini-2.5-flash` [20].
- Perfil técnico `university`, correspondiente a `openai/gpt-oss-20b` [21] servido mediante vLLM en el entorno universitario.

En todos los casos el workflow usa el modelo configurado para interpretación de consulta, generación de código e interpretación final. Si falta una clave, el proveedor no está disponible, el código se rechaza o la ejecución falla, el caso debe quedar registrado como error controlado.

8.2. Métricas y criterios

La evaluación técnica registrará:

- **Casos completados:** consultas válidas que terminan con estado `completed`.
- **Errores controlados:** consultas inválidas o fallidas que terminan con explicación.
- **Errores LLM:** problemas de configuración, red, cuota, disponibilidad, contexto o formato.
- **Código rechazado:** scripts que no superan el contrato de seguridad.
- **Errores de ejecución:** scripts válidos que fallan al calcular o serializar la salida.
- **Violaciones de recomendación de inversión:** presencia de recomendaciones o predicciones fuera del alcance del MVP.
- **Tiempo de ejecución:** duración observada por caso y proveedor.

Estas métricas deben complementarse con revisión humana. La completitud técnica no garantiza por sí sola que el análisis sea útil. La revisión cualitativa valora comprensión de la consulta, selección de métricas, suficiencia del código generado, adecuación de tablas o gráficas, exactitud respecto a los datos ejecutados, claridad del texto final, reconocimiento de limitaciones y ausencia de recomendaciones de inversión.

Esta decisión es coherente con la necesidad de explicar cómo se calcula y presenta el rendimiento histórico [14], y con la separación entre retorno, riesgo, volatilidad y correlación como conceptos de análisis histórico [15], [16].

La rúbrica se define en `docs/evaluacion_cualitativa_llm.md`. En ella se separan criterios del primer agente, como selección de métricas y generación de datos suficientes, y criterios del segundo agente, como claridad, prudencia y fidelidad a la salida ejecutada.

8.3. Niveles cualitativos de salida

La evaluación se estructura en tres niveles:

- **Nivel A:** consultas simples sin requisitos de presentación. La respuesta esperada es breve y debe incluir métricas básicas suficientes.
- **Nivel B:** consultas que piden un análisis más completo. La respuesta esperada incluye texto, tabla de métricas y una gráfica principal o datos estructurados para generarla.
- **Nivel C:** consultas profesionales o detalladas. La respuesta esperada incluye varias salidas estructuradas, por ejemplo tabla, evolución normalizada, drawdown, clasificación mensual, desglose de riesgo o conclusión ejecutiva.

Para series temporales se priorizan gráficos de línea y, cuando se comparan activos con escalas distintas, una evolución normalizada permite comparar la tendencia sin confundir niveles de precio absolutos. También se evita abusar de demasiadas series o ejes dobles, porque pueden dificultar la lectura [18]. En consultas de riesgo se considera especialmente útil el drawdown, ya que resume caídas históricas desde máximos previos y se entiende con facilidad [17].

Nivel	Uso previsto	Salida esperada
A	Query simple sin formato específico	Texto breve y métricas básicas en tabla o bloque estructurado.
B	Análisis más completo o comparativa clara	Texto, tabla de métricas y una gráfica principal o datos para representarla.
C	Petición profesional, detallada o con varias salidas	Texto, tablas, varias visualizaciones o datos estructurados, conclusión y limitaciones.

Tabla 8.1: Niveles cualitativos de profundidad del análisis.

8.4. Batería progresiva

La batería cualitativa incluye consultas A/B/C sobre crecimiento, comparación de activos, visión descriptiva, retornos, riesgo histórico y análisis técnico. También se mantiene una query de estrés visual que obliga al sistema a respetar cuatro bloques de salida:

Compara QQQ y SPY durante 2024 como si fuera un informe para un cliente no técnico. Quiero que me muestres los datos exactamente en cuatro bloques: 1) una tabla resumen con rentabilidad total, volatilidad anualizada, máximo drawdown, mejor mes y peor mes; 2) una tabla de clasificación mensual indicando qué activo ganó cada mes; 3) datos para una gráfica normalizada base 100 de ambos activos; 4) datos para una gráfica de drawdown. Cierra con una conclusión clara, sin recomendar comprar ni vender.

Esta consulta comprueba simultáneamente comprensión, selección de métricas, generación de código, control de formato, datos para visualización e interpretación final. La batería completa y las familias técnicas complementarias se describen en `docs/planificacion_evaluacion_tfm.md`.

8.5. Protocolo de ejecución definitiva

La evaluación final seguirá los siguientes pasos:

1. Fijar la batería exacta de consultas y los CSV locales utilizados.
2. Ejecutar la misma batería con cada perfil LLM seleccionado.
3. Guardar por caso el plan, el código, la salida estructurada, los avisos, el estado y el tiempo.
4. Revisar cada respuesta con la plantilla manual definida previamente.
5. Separar fallos de proveedor, fallos de codegen, rechazos de seguridad, fallos de ejecución y defectos de interpretación.
6. Presentar la comparación como evidencia observada bajo condiciones concretas, no como clasificación estadística absoluta.

8.6. Resultados pendientes

Los resultados comparativos definitivos se incorporarán después de ejecutar la batería cerrada. Hasta entonces no se presentan porcentajes de completitud, latencias por proveedor ni ejemplos numéricos como conclusiones finales del TFM.

Los informes Markdown y JSON producidos por scripts de evaluación se consideran artefactos temporales mientras no se haya seleccionado la pasada definitiva. Esta separación evita confundir registros de depuración con evidencia final.

8.7. Limitaciones metodológicas

La calidad de las respuestas LLM requiere revisión humana y las cifras de latencia dependen de red, cuota y carga del proveedor. Además, una batería reducida no permite afirmar robustez estadística. La comparación debe entenderse como exploratoria, trazable y revisable.

El sistema trabaja con datos históricos locales. Su objetivo es describir métricas observadas, no producir asesoramiento financiero, incorporar noticias externas ni predecir el comportamiento futuro de los activos.

8.8. Síntesis

La evaluación propuesta combina criterios técnicos y cualitativos. Su aportación principal consiste en revisar por separado interpretación, generación de código, validación, ejecución e interpretación final. Esta estructura permite analizar no solo si el workflow termina, sino también si responde con datos suficientes, comprensibles y trazables.

Capítulo 9

Aspectos éticos, seguridad y uso responsable

9.1. Delimitación financiera del sistema

El sistema se orienta exclusivamente al análisis financiero histórico y descriptivo. Sus respuestas deben interpretarse como explicaciones basadas en datos pasados, no como predicciones ni como recomendaciones de inversión. Esta delimitación forma parte del diseño del MVP y debe mantenerse visible tanto en la memoria como en cualquier interfaz futura.

El prototipo calcula métricas como crecimiento, retornos, volatilidad, drawdown e indicadores técnicos básicos. No realiza predicciones, no recomienda comprar o vender activos y no adapta decisiones a perfiles personales de riesgo.

9.2. Riesgos de interpretación

Un riesgo relevante es que el usuario confunda una explicación descriptiva con asesoramiento financiero. Para reducirlo, el sistema debe utilizar formulaciones prudentes, explicitar el período analizado y recordar que los resultados históricos no garantizan comportamientos futuros.

La capa LLM aumenta este riesgo si redacta respuestas demasiado persuasivas. Por ello, la configuración actual pide al modelo interpretar resultados ya calculados, no añadir datos externos y evitar recomendaciones de inversión.

9.3. Seguridad en la generación y ejecución de código

La generación automática de código introduce riesgos específicos. Un script mal generado podría fallar, calcular una métrica incorrecta o acceder a recursos no previs-

tos. En este proyecto, el riesgo se reduce mediante entradas estructuradas, ejecución controlada, validación mínima del contrato y captura explícita de salidas.

El código generado se ejecuta como proceso externo, con registro de salida estándar, salida de error y código de retorno. Esta estrategia permite revisar errores y evita que la respuesta final oculte fallos de ejecución.

9.4. Privacidad y datos

Los datos utilizados son series históricas de mercado y no contienen datos personales. Aun así, las claves de API de proveedores LLM se consideran información sensible y se almacenan exclusivamente en el fichero local `.env`, excluido del control de versiones.

En una versión productiva, sería necesario revisar las condiciones de uso de los proveedores de datos y de modelos, así como las políticas de retención de información enviada a APIs externas.

9.5. Sostenibilidad, costes y recursos

El enfoque incremental reduce costes computacionales. El MVP no entrena modelos desde cero y puede ejecutarse en CPU sobre datos ya descargados. La capa LLM se concentra principalmente en interpretación y permite comparar proveedores sin entrenar ni ajustar pesos.

Esta estrategia también mejora la sostenibilidad del desarrollo: evita ejecuciones innecesarias de modelos grandes, reduce dependencia de infraestructura GPU y permite comparar proveedores antes de elegir una configuración de uso continuado.

9.6. Limitaciones de uso

El sistema no debe utilizarse como herramienta profesional de decisión financiera sin validaciones adicionales. Su valor actual es académico y experimental: demostrar una arquitectura modular, trazable y robusta para automatizar análisis históricos.

Entre las limitaciones principales se encuentran el uso de un conjunto de datos acotado, la dependencia de datos obtenidos desde Yahoo Finance mediante `yfinance` [12], [13], la ausencia de validación financiera profesional y la no incorporación de información fundamental, noticias o variables macroeconómicas.

Capítulo 10

Conclusiones y trabajo futuro

El trabajo desarrollado demuestra la viabilidad de un sistema LLM para análisis financiero histórico basado en una arquitectura modular, trazable y extensible. El MVP valida entradas estructuradas, interpreta la intención del usuario, genera código Python, revisa su seguridad, ejecuta el script sobre CSV locales y redacta una respuesta final a partir de resultados observados. La aportación principal no es predecir el mercado, sino conectar lenguaje natural, cálculo reproducible y explicación financiera prudente.

Los objetivos planteados se cumplen en una versión funcional y evaluable. Se ha definido un workflow modular, se ha incorporado generación de código mediante LLM, se ha añadido una capa de seguridad previa a la ejecución, se han conservado artefactos auditables y se han preparado scripts de evaluación con perfiles Groq, Gemini y university. Además, se ha ampliado la evaluación desde consultas simples hacia una metodología cualitativa por niveles A/B/C, más cercana a la forma real en que un usuario valora una respuesta de análisis.

Una aportación importante es la separación de responsabilidades. El primer agente debe producir código, métricas, tablas o datos de visualización suficientes para responder la consulta. El segundo agente debe interpretar esos resultados con claridad, distinguir dato histórico, lectura y limitaciones, y evitar recomendaciones de inversión. Esta división permite revisar el sistema por partes y detectar si un fallo procede de la comprensión de la query, del código, de la ejecución o de la interpretación final.

La evaluación realizada muestra un estado prometedor, pero todavía experimental. En la pasada progresiva actualizada del 28/05/2026 se observaron tres resultados principales: Groq con `llama-3.3-70b-versatile` [19] completó cinco de cinco casos; el perfil universitario basado en `openai/gpt-oss-20b` [21] también completó cinco de cinco casos, incluyendo Nivel C y la query de estrés visual; y Gemini con `gemini-2.5-flash` [20] completó cuatro de cinco casos, fallando solo en la query visual más exigente por generación de código. Por tanto, no se afirma una superioridad objetiva definitiva de ningún proveedor; se presenta evidencia observada bajo condiciones concretas.

Los resultados también han permitido mejorar el diseño. Las consultas profesionales o visuales generaban salidas muy extensas, especialmente cuando incluían series temporales completas para gráficas. Esto reveló un problema práctico de contexto para el segundo agente. La solución incorporada compacta la información enviada al LLM interpretador, manteniendo la salida completa como artefacto auditable. Este ajuste refuerza la robustez del MVP sin perder trazabilidad.

El prototipo trabaja con un conjunto acotado de datos históricos ya descargados. No aborda predicción futura, recomendación de inversión, análisis fundamental ni incorporación de noticias. La generación de código mediante LLM queda limitada por contratos, validaciones y revisión posterior, por lo que requiere más pruebas antes de cualquier uso fuera del entorno académico. La revisión cualitativa también incorpora juicio humano, aunque se controla mediante criterios explícitos y una plantilla común.

Como trabajo futuro se proponen varias líneas: ampliar la batería de consultas de estrés, repetir la comparación en varias rondas para reducir el efecto de cuotas y disponibilidad puntual, reforzar el análisis de seguridad del código generado, registrar experimentos con herramientas MLOps, automatizar parte de la rúbrica cualitativa sin sustituir la revisión humana y mejorar la generación de visualizaciones exportables. También sería natural incorporar pruebas con más activos, ventanas temporales y perfiles de usuario, manteniendo siempre la separación entre rendimiento histórico observado e interpretación.

En conjunto, el TFM se encuentra en un punto sólido para memoria: existe un MVP ejecutable, una metodología defendible, evidencias guardadas, comparación exploratoria entre modelos y ejemplos visuales que muestran cómo se ha aumentado progresivamente la dificultad de las consultas.

Referencias

- [1] Analytics Vidhya, *60+ Generative AI Terms You Must Know*, 2024. visitado 25 de mayo de 2026. dirección: <https://www.analyticsvidhya.com/blog/2024/01/generative-ai-terms/>
- [2] D. Araci, «FinBERT: Financial Sentiment Analysis with Pre-trained Language Models,» *arXiv preprint arXiv:1908.10063*, 2019. DOI: 10.48550/arXiv.1908.10063 arXiv: 1908.10063 [cs.CL]. dirección: <https://arxiv.org/abs/1908.10063>
- [3] S. Wu et al., «BloombergGPT: A Large Language Model for Finance,» *arXiv preprint arXiv:2303.17564*, 2023. DOI: 10.48550/arXiv.2303.17564 arXiv: 2303.17564 [cs.LG]. dirección: <https://arxiv.org/abs/2303.17564>
- [4] H. Yang, X.-Y. Liu y C. D. Wang, «FinGPT: Open-Source Financial Large Language Models,» *arXiv preprint arXiv:2306.06031*, 2023. DOI: 10.48550/arXiv.2306.06031 arXiv: 2306.06031 [q-fin.CP]. dirección: <https://arxiv.org/abs/2306.06031>
- [5] S. Yao et al., «ReAct: Synergizing Reasoning and Acting in Language Models,» en *International Conference on Learning Representations*, 2023. arXiv: 2210.03629 [cs.CL]. dirección: <https://arxiv.org/abs/2210.03629>
- [6] T. Schick et al., «Toolformer: Language Models Can Teach Themselves to Use Tools,» *arXiv preprint arXiv:2302.04761*, 2023. DOI: 10.48550/arXiv.2302.04761 arXiv: 2302.04761 [cs.CL]. dirección: <https://arxiv.org/abs/2302.04761>
- [7] L. Wang et al., «A Survey on Large Language Model based Autonomous Agents,» *arXiv preprint arXiv:2308.11432*, 2023. DOI: 10.48550/arXiv.2308.11432 arXiv: 2308.11432 [cs.AI]. dirección: <https://arxiv.org/abs/2308.11432>
- [8] M. Chen et al., «Evaluating Large Language Models Trained on Code,» *arXiv preprint arXiv:2107.03374*, 2021. DOI: 10.48550/arXiv.2107.03374 arXiv: 2107.03374 [cs.LG]. dirección: <https://arxiv.org/abs/2107.03374>

- [9] Y. Lai et al., «DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation,» *arXiv preprint arXiv:2211.11501*, 2022. DOI: 10.48550/arXiv.2211.11501 arXiv: 2211.11501 [cs.SE]. dirección: <https://arxiv.org/abs/2211.11501>
- [10] Z. Ji et al., «Survey of Hallucination in Natural Language Generation,» *ACM Computing Surveys*, 2022. DOI: 10.1145/3571730 arXiv: 2202.03629 [cs.CL]. dirección: <https://arxiv.org/abs/2202.03629>
- [11] P. Lewis et al., «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,» en *Advances in Neural Information Processing Systems*, 2020. arXiv: 2005.11401 [cs.CL]. dirección: <https://arxiv.org/abs/2005.11401>
- [12] R. Aroussi, *yfinance Python Library*, 2026. visitado 25 de mayo de 2026. dirección: <https://github.com/ranaroussi/yfinance>
- [13] Yahoo Finance Help, *Download historical data in Yahoo Finance*, 2026. visitado 25 de mayo de 2026. dirección: <https://help.yahoo.com/kb/finance/historical-prices-adjusted-close-sln2311.html>
- [14] U.S. Securities and Exchange Commission, *Investor Bulletin: Performance Claims*, 2022. visitado 27 de mayo de 2026. dirección: <https://www.investor.gov/introduction-investing/general-resources/news-alerts/alerts-bulletins/investor-bulletins-47>
- [15] Investor.gov, *What is Risk?* 2026. visitado 27 de mayo de 2026. dirección: <https://www.investor.gov/introduction-investing/investing-basics/what-risk>
- [16] CFA Institute, *Portfolio Risk and Return: Part I*, 2026. visitado 27 de mayo de 2026. dirección: <https://www.cfainstitute.org/insights/professional-learning/refresher-readings/2026/portfolio-risk-return-part-1>
- [17] CFA Institute, *Performance Drawdowns in Asset Management: Extending Drawdown Analysis to Active Returns*, 2018. visitado 27 de mayo de 2026. dirección: <https://rpc.cfainstitute.org/research/cfa-digest/2018/01/dig-v48-n1-3>
- [18] Atlassian, *A Complete Guide to Line Charts*, 2026. visitado 27 de mayo de 2026. dirección: <https://www.atlassian.com/data/charts/line-chart-complete-guide>
- [19] Groq, *Llama 3.3 70B Model Documentation*, 2026. dirección: <https://console.groq.com/docs/model/llama-3.3-70b-versatile>

- [20] Google Cloud, *Gemini 2.5 Flash Model Documentation*, 2025. dirección: <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-5-flash>
- [21] OpenAI, *gpt-oss-120b & gpt-oss-20b Model Card*, 2025. DOI: 10.48550/arXiv.2508.10925 arXiv: 2508.10925 [cs.CL]. dirección: <https://arxiv.org/abs/2508.10925>
- [22] LangChain, «LangGraph: Building Stateful, Multi-Actor Applications with LLMs,» 2024. dirección: <https://langchain-ai.github.io/langgraph/>
- [23] T. Zhou, P. Wang, Y. Wu y H. Yang, «FinRobot: AI Agent for Equity Research and Valuation with Large Language Models,» *arXiv preprint arXiv:2411.08804*, 2024. DOI: 10.48550/arXiv.2411.08804 arXiv: 2411.08804 [q-fin.CP]. dirección: <https://arxiv.org/abs/2411.08804>
- [24] Y. Xiao, E. Sun, D. Luo y W. Wang, «TradingAgents: Multi-Agents LLM Financial Trading Framework,» *arXiv preprint arXiv:2412.20138*, 2024. DOI: 10.48550/arXiv.2412.20138 arXiv: 2412.20138 [q-fin.TR]. dirección: <https://arxiv.org/abs/2412.20138>
- [25] Groq, *Groq API Documentation*, 2026. dirección: <https://console.groq.com/docs>
- [26] Meta, *Llama 3.3 70B Instruct Model Card*, 2024. dirección: https://github.com/meta-llama/llama-models/blob/main/models/llama3_3/MODEL_CARD.md
- [27] AI@Meta, «The Llama 3 Herd of Models,» *arXiv preprint arXiv:2407.21783*, 2024. DOI: 10.48550/arXiv.2407.21783 arXiv: 2407.21783 [cs.AI]. dirección: <https://arxiv.org/abs/2407.21783>
- [28] Google AI for Developers, *Gemini API Documentation*, 2026. dirección: <https://ai.google.dev/gemini-api/docs>
- [29] Gemini Team, Google, «Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities,» Google, inf. téc., 2025. dirección: https://storage.googleapis.com/deepmind-media/gemini/gemini_v2_5_report.pdf
- [30] OpenAI, *OpenAI o1 System Card*, 2024. DOI: 10.48550/arXiv.2412.16720 arXiv: 2412.16720 [cs.AI]. dirección: <https://arxiv.org/abs/2412.16720>
- [31] DeepSeek-AI, «DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,» *arXiv preprint arXiv:2501.12948*, 2025. DOI: 10.48550/arXiv.2501.12948 arXiv: 2501.12948 [cs.CL]. dirección: <https://arxiv.org/abs/2501.12948>

Apéndice A

Anexos

A.1. Ejecución del MVP

El proyecto puede ejecutarse con perfiles LLM mediante los ejemplos incluidos en el repositorio:

```
python run_mvp.py --example growth_nvda
python run_mvp.py --example compare_nvda_amd
python run_mvp.py --example overview_aapl
python run_mvp.py --example returns_qqq_spy
python run_mvp.py --example risk_qqq_spy
python run_mvp.py --example technical_aapl
```

Además de estos ejemplos funcionales, se prepararon scripts específicos para la evaluación cualitativa, la demostración para cliente y la comparación progresiva entre proveedores LLM:

```
python scripts\generate_qualitative_client_demo.py
python scripts\generate_qualitative_demo_report.py
python scripts\evaluate_progressive_llm_scope.py ^
    --profiles groq gemini university
```

La última orden puede restringirse a una muestra concreta de dificultad creciente:

```
python scripts\evaluate_progressive_llm_scope.py ^
    --profiles groq gemini university ^
    --cases level_a_nvda_growth level_a_qqq_spy_returns ^
        level_b_qqq_spy_clear_compare ^
        level_c_qqq_spy_professional ^
        stress_visual_qqq_spy_monthly
```

A.2. Configuración de APIs

Las claves de API se guardan localmente en el fichero `.env`, que no debe subirse al repositorio. El proyecto asume llamadas LLM en el flujo principal, por lo que no hay variables de activación.

Para activar Groq con `llama-3.3-70b-versatile` [19], modelo Llama 3.3 70B Instruct orientado a razonamiento general y baja latencia, la configuración es:

```
llama-3.3-70b-versatile [19]
```

```
LLM_PROFILE=groq  
GROQ_MODEL=llama-3.3-70b-versatile
```

Para activar Gemini con `gemini-2.5-flash` [20], modelo Flash de Gemini 2.5 orientado a rapidez y razonamiento, la configuración es:

```
gemini-2.5-flash [20]
```

```
LLM_PROFILE=gemini  
GEMINI_MODEL=gemini-2.5-flash
```

Para activar `openai/gpt-oss-20b` [21] servido por vLLM en el entorno universitario, modelo abierto de la familia gpt-oss usado por su capacidad de razonamiento, seguimiento de instrucciones y generación de código, el perfil técnico configurado es `university`:

```
openai/gpt-oss-20b [21]
```

```
LLM_PROFILE=university  
UNIVERSITY_BASE_URL=https://w1.etsisi.upm.es/vllm/v1  
UNIVERSITY_MODEL=openai/gpt-oss-20b
```

A.3. Batería de consultas A/B/C

La batería cualitativa se diseñó para comprobar que el sistema no solo termina la ejecución, sino que adapta la profundidad del análisis a la consulta. Las seis primeras consultas corresponden al Nivel A y representan peticiones habituales sin formato especial. Las tres siguientes elevan la exigencia al Nivel B, y las tres últimas obligan a una salida de Nivel C.

Tabla A.1: Batería cualitativa de consultas por nivel de profundidad.

ID	Consulta	Nivel	Objetivo de evaluación
A1	Analiza el crecimiento de NVDA durante los últimos cinco años.	A	Rentabilidad y resumen simple.
A2	Dime la rentabilidad de AAPL en el período disponible.	A	Cálculo directo y respuesta breve.
A3	Resume la evolución de SPY en 2024.	A	Visión general sin gráficas obligatorias.
A4	Compara de forma sencilla QQQ y SPY en 2024.	A	Comparación básica de retornos.
A5	Indica si AAPL fue volátil en los últimos tres meses disponibles.	A	Uso de volatilidad y drawdown.
A6	Resume el volumen negociado de NVDA en 2024.	A	Selección de métrica de volumen.
B1	Hazme un análisis completo de NVDA con una tabla de métricas y una gráfica de precio.	B	Tabla y visualización principal.
B2	Compárame QQQ y SPY de forma clara con evolución normalizada.	B	Comparación relativa base 100.
B3	Quiero entender retorno y riesgo de AAPL en el último año.	B	Relación riesgo-retorno.
C1	Dame un análisis profesional de QQQ y SPY con tabla, gráfica normalizada, drawdown y conclusión.	C	Varias salidas estructuradas.
C2	Compara NVDA y AMD con rentabilidad, volatilidad, correlación, mejor y peor mes, y explicación para usuario no técnico.	C	Múltiples métricas y adaptación de lenguaje.
C3	Analiza SPY desde 2020 con retornos anuales, drawdown, medias móviles y limitaciones del análisis.	C	Desglose temporal y limitaciones.

A.4. Query de estrés visual

Para forzar al sistema a respetar una estructura concreta se incorporó una query adicional de estrés visual:

Compara QQQ y SPY durante 2024 como si fuera un informe para un cliente no técnico. Quiero que me muestres los datos exactamente en cuatro bloques: 1) una tabla resumen con rentabilidad total, volatilidad anualizada, máximo drawdown, mejor mes y peor mes; 2) una tabla de clasificación mensual indicando qué activo ganó cada mes; 3) datos para una gráfica normalizada base 100 de ambos activos; 4) datos para una gráfica de drawdown. Cierra con una conclusión clara, sin recomendar comprar ni vender.

Esta consulta se eligió porque comprueba simultáneamente comprensión, selección de métricas, generación de código, control de formato, datos para visualización e interpretación final sin recomendación de inversión.

A.5. Documentación estable

La documentación Markdown estable conserva el diseño, la metodología y los aprendizajes reutilizables. Los informes producidos por ejecuciones concretas se tratan como artefactos temporales y se regenerarán cuando se realice la evaluación definitiva:

- `docs/evolucion_mvp_tfm.md`: evolución incremental del MVP desde dos intencionalidades hasta el flujo actual con codegen controlado.
- `docs/arquitectura_llm_codegen.md`: descripción técnica del flujo LLM, nodos, prompts, seguridad y ejecución.
- `docs/llm_api_setup.md`: configuración de perfiles Groq, Gemini y servidor vLLM universitario.
- `docs/planificacion_evaluacion_tfm.md`: protocolo de evaluación definitiva, familias de consultas, aprendizajes técnicos y líneas de trabajo futuro.
- `docs/control_tamano_salidas_visualizacion.md`: justificación de la compactación de salidas visuales largas antes de la interpretación final.
- `docs/evaluacion_cualitativa_llm.md`: metodología cualitativa, criterios, niveles A/B/C y rúbrica.

A.6. Plantilla de revisión manual

La siguiente plantilla se usa para revisar cada ejecución sin presentar la evaluación como una métrica matemática absoluta:

Criterio	Valoración	Observaciones
Comprensión de la consulta	Excelente / Correcto / Parcial / Deficiente	
Selección de métricas y datos	Excelente / Correcto / Parcial / Deficiente	
Código generado	Excelente / Correcto / Parcial / Deficiente	
Tablas y gráficas solicitadas	Excelente / Correcto / Parcial / Deficiente	
Exactitud frente a la salida ejecutada	Excelente / Correcto / Parcial / Deficiente	
Claridad del análisis final	Excelente / Correcto / Parcial / Deficiente	
Utilidad para el usuario	Excelente / Correcto / Parcial / Deficiente	
Reconocimiento de limitaciones	Excelente / Correcto / Parcial / Deficiente	
Ausencia de recomendación de inversión	Excelente / Correcto / Parcial / Deficiente	
Trazabilidad consulta-código-resultados	Excelente / Correcto / Parcial / Deficiente	

Tabla A.2: Plantilla de revisión manual cualitativa.

A.7. Documentación interna de apoyo

Durante el desarrollo se conservaron notas técnicas y documentos metodológicos en Markdown. Estos documentos no sustituyen a la memoria, pero permiten reconstruir decisiones de diseño, preparar la evaluación final y registrar limitaciones conocidas sin mezclar esos contenidos con salidas provisionales.

Documento interno	Uso principal en la memoria
docs/evolucion_mvp_tfm.md	Motivación y metodología incremental del prototipo.
docs/arquitectura_llm_codegen.md	Diseño del sistema, nodos, contrato de codegen y trazabilidad.
docs/control_tamano_salidas_visualizacion.md	Compactación de salidas largas y gestión de contexto en Nivel B/C.
docs/evaluacion_cualitativa_llm.md	Criterios de buen análisis, niveles A/B/C y plantilla de revisión.
docs/planificacion_evaluacion_tfm.md	Protocolo definitivo, aprendizajes técnicos, familias de consultas y trabajo futuro.

Tabla A.3: Relación entre documentación Markdown y secciones de la memoria.